Live Bilingual Classroom Translation System

Real-time English-to-Chinese Classroom Translation

Live Bilingual Classroom Translation System

Project: Real-time English-to-Chinese Classroom Translation

Abstract This report documents the design and implementation of the real-time bilingual classroom translation system developed in the script light_audio_llm.py. The system captures microphone input, performs speech-to-text conversion, corrects transcription errors in context, translates corrected English into Chinese, and displays the original text and translated output in near-real time. The design targets classroom settings where latency must remain close to three seconds, while maintaining acceptable robustness under realistic acoustic noise and speech variability.

The central engineering principle is to balance accuracy, latency, privacy, and cost. For a classroom application, small local models are preferable when they meet the performance target and avoid internet dependency. In this design, faster-whisper handles transcription locally, MarianMT performs translation locally, and a compact local LLM is used only for lightweight correction. This combination reduces network delays and avoids exposing classroom speech to third-party cloud services.

System Architecture Microphone -> VAD / Energy Gate -> Local STT -> Context-aware correction -> Local translation -> Dual-line display

The implementation in light_audio_llm.py follows this pipeline. Audio is captured continuously from the default microphone, a lightweight voice activity detection gate removes silence and low-energy noise, the system transcribes spoken English, applies correction based on previous classroom context, translates the corrected sentence into Chinese, and prints the outputs in the terminal or GUI. In the desktop version, the results are displayed in separate panels using a Tkinter interface.

1. Audio capture The project uses the computer's built-in microphone as the default input because the system is designed for low-cost classroom prototyping and portability. This is the most practical option for a student project because it requires no extra hardware, no additional USB audio interface, and no complicated setup. The script automatically obtains the default input device through PyAudio and then tries several common channel/rate combinations (2-channel and 1-channel audio, 44.1 kHz, 48 kHz, and 16 kHz) until a valid stream is opened.

Alternative options included an external wireless microphone or a USB microphone. An external microphone would improve signal quality and reduce handling noise in a noisy room, but it introduces extra setup complexity, cost, and dependence on hardware availability. For portability and reproducibility, the built-in laptop microphone is the better trade-off. In regular classroom settings, the built-in mic is sufficient for a prototype, especially when combined with VAD and energy thresholding. It also allows the application to run on many laptops without requiring the user to install specialized hardware.

The main disadvantage is sound quality and robustness. Built-in laptop microphones often capture room reverberation, fan noise, keyboard taps, and background chatter. However, the design tolerates this by using a narrow 30 ms frame size, resampling to 16 kHz, and applying gating logic before transcription. This means the decision to use the built-in microphone is a practical engineering choice rather than an ideal acoustic choice.

2. Noise handling The system uses a combination of WebRTC VAD and a simple amplitude-based energy gate. The main reason is that the project needs real-time responsiveness without building a full denoising pipeline. We use webrtcvad.Vad(2) to decide whether a frame contains speech, and we also compute the root-mean-square energy of the audio signal. If the signal energy is lower than a small threshold, the code discards it as near-silence or background noise. This prevents the model from wasting compute on low-quality frames and reduces false positives caused by breathing, keyboard noise, or room hum.

This method is selected over a heavy denoising pipeline because the application must stay within the three-second end-to-end latency target. Deep denoisers would improve signal quality but would significantly increase model time and memory cost. In a classroom environment, a light VAD + energy check is a more realistic engineering choice. It is also easier to optimize in Python with small latency cost.

The project tests noisy conditions by using a classroom-style input scenario: room chatter, small fan noise, speaker movement, and occasional intermittent background sounds. The system is designed to isolate speech segments by requiring a continuous speech segment to pass the VAD gate, then waiting for a silence threshold before triggering recognition. The logic is intentionally conservative: a short speech burst without enough content is discarded, while longer segments are sent to the transcription stage.

This is a practical form of noise handling. It does not guarantee full robustness in highly noisy environments, but it is sufficient for a prototype and maintains the real-time requirement. For more severe environments, a dedicated noise suppression front end such as RNNoise or a beamforming microphone could be added later.

3. Transcription The transcription stage uses faster-whisper with the base.en English model. This is a local model and is the strongest choice for a classroom real-time system because it provides low

latency, strong transcription quality, and privacy preservation. The script loads WhisperModel("base.en", device="cpu", compute_type="int8"), which keeps all audio processing on the local machine rather than sending speech to an external service.

Compared with OpenAI or cloud transcription APIs, the local Whisper approach has several important advantages: - Zero recurring cost: there is no API usage fee per minute of transcription. - Low latency: local inference is fast on CPU and avoids network round-trip delay. - Privacy: classroom speech never leaves the device, which is important in educational contexts. - Internet independence: the system works even when the network is weak or unavailable. - Simpler testing: no API key or authentication is required.

By contrast, cloud APIs such as OpenAI Whisper or Azure Speech generally offer excellent accuracy and can be stronger in very challenging acoustic environments, but they also add network delay, internet dependency, cost, and privacy concerns. In many teaching scenarios, students and instructors may not want classroom recordings transmitted to a remote service. For these reasons, local faster-whisper is the better engineering decision for this project.

The trade-off is that cloud APIs may still be superior in borderline cases with heavy accent variation or noisy classrooms. However, the assignment constraint of a low-latency system and the need for a local, privacy-respecting deployment strongly favor the local Whisper approach.

4. Correction The correction stage compares two major strategies: a local small model and a cloud-scale model. In the current implementation, the system uses a lightweight local correction path through Ollama with the model qwen2.5:0.5b. This local model is called only when the transcription contains likely errors or unnatural phrasing. If the correction service fails or times out, the system falls back to a rule-based cleanup procedure.

A cloud large model such as GPT-4 would be stronger at contextual correction and rewriting. It could detect more subtle errors, correct grammar, and transform awkward spoken phrases into cleaner classroom English. In theory, this leads to better final text quality. However, it also introduces several problems: network delay, latency variance, cost per request, and privacy concerns because speech content is sent externally.

For a live classroom application, the stronger constraint is not merely textual quality, but delivery time. The project specification requires delay around three seconds or less. A cloud-based LLM could easily push the total end-to-end delay above that limit because the request must travel to a remote service, wait for processing, and return. A small local model is therefore preferable because it preserves responsiveness while still improving common transcription mistakes.

The project uses qwen2.5:0.5b because it delivers enough contextual correction quality for classroom phrases without dominating the total latency budget. In addition, if the model is unavailable, the fallback replacement dictionary protects the system from crashing. This is a sensible design choice because it ensures robustness and continuous operation under limited hardware or service conditions.

5. Translation For translation, the implementation uses a separate local HuggingFace model: Helsinki-NLP/opus-mt-en-zh, which is a MarianMT encoder-decoder model. This is a clearer design than assigning translation to the same LLM used for correction. The reason is speed and specialization.

A general-purpose LLM can translate text, but it is usually slower and less deterministic than a dedicated machine translation model. In this project, the translation model is large compared with a rule-based or small model, but it is highly optimized for bilingual conversion and is significantly faster than sending each sentence to a general-purpose LLM. In practice, on CPU, MarianMT translation often responds in approximately 0.1 to 0.2 seconds per short sentence, which is very competitive for a live system.

This is a key design decision. The pipeline benefits from a specialized model for translation and a different model for correction. The correction model can focus on grammar and classroom context, while the translation model focuses on accurate English-to-Chinese conversion. This separation reduces unnecessary load on the LLM and allows the system to meet the strict latency target.

The alternative of using a single LLM for both correction and translation was considered, but it would cause more latency and less clear optimization. Since the project required real-time classroom support, a dedicated translation model is the stronger engineering choice.

6. User interface The user interface is implemented in Python Tkinter. This is a deliberate choice because the system is intended for rapid prototype development and because the current environment is a local classroom demonstration. Tkinter is a lightweight GUI library, requires no external browser or frontend framework, and can be run quickly on Linux, Windows, and macOS with minimal setup requirements.

The alternatives considered were a native macOS interface and a web-based interface. A native macOS UI would offer a polished and responsive look, but it would be platform-specific and would require a different toolchain. It would also be harder to demonstrate across multiple machines in the classroom. A web interface could provide a modern design and cross-platform compatibility, but it introduces additional browser and network concerns. For a student project, a web interface would require a server, browser permissions, and more infrastructure for live audio streaming.

Tkinter is suitable because it allows a simpler architecture: the same Python process handles audio capture, transcription, correction, translation, and rendering. This leads to lower development complexity and easier debugging. It is not the most elegant UI for a commercial deployment, but for a live project demonstration it is fast, practical, and cross-platform. This is the right engineering choice for the project stage.

7. Media and data formats The application is designed around live streaming audio, but the underlying signal processing is carefully

normalized to a consistent format. The code converts audio to 16 kHz, mono (when necessary), and 16-bit PCM. The system resamples input audio if the hardware sample rate differs, converts stereo to mono by averaging left and right channels, pads or truncates frames to a fixed chunk size, and converts the result to int16 bytes for VAD and Whisper processing.

This standardization step is essential. Whisper performs best when the input is regular and predictable. By converting the stream into a fixed 16 kHz mono PCM representation, the model receives a cleaner and more consistent signal. The use of 30 ms chunks is also compatible with low-latency streaming and VAD detection.

If the project later supports recorded media files, ffmpeg can be used to normalize files before transcription. A typical preprocessing pipeline would be: input file -> ffmpeg decode -> resample to 16 kHz mono -> PCM or WAV export -> send to Whisper. This would allow the system to handle classroom recordings, lecture replays, or offline evaluation with the same processing assumptions as the live pipeline. Even without file support, the same standardization logic is applied in real time, which keeps the design consistent and robust.

Testing and validation approach The system is tested incrementally in accordance with the project pipeline. First, microphone input is confirmed, then a short transcription is validated, then the system measures transcription delay and translation delay. Additional testing is performed with classroom-like background noise, using the speech activity detection threshold to prevent silence and low-energy noise from being processed. The project also evaluates the end-to-end delay by timing the STT stage and the translation stage separately. The measured values are then reported to assess whether the application remains within the target latency constraint.

The main strengths of the implemented design are its responsiveness, local deployment, and minimal infrastructure requirements. The main limitations are that classroom noise can still reduce accuracy, the correction stage relies on a small local model, and a very loud acoustic environment may still be hard to handle without external microphones or dedicated denoising hardware.

Conclusion The system in light_audio_llm.py is an engineering compromise that prioritizes low latency, privacy, and simplicity without sacrificing usability. The selected design uses the built-in microphone for portability, WebRTC VAD and energy gating for noise handling, local faster-whisper for transcription, a compact local LLM for correction, MarianMT for translation, Tkinter for the prototype interface, and a standardized audio pipeline for stable processing. These choices are well aligned with the assignment requirements and with the practical realities of a live classroom translation application.